LSTM 장단기 메모리 (Long Short-Term Memory)

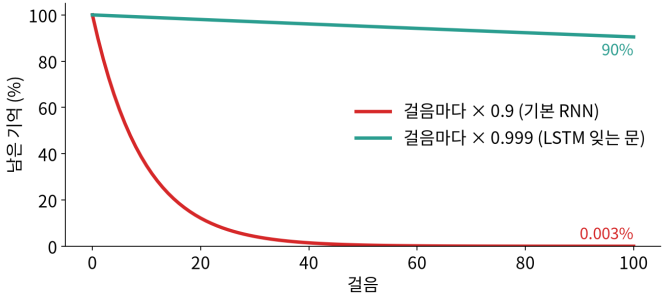
곱하기 대신 더하기로 기억을 지킨다

Machine Learning and Deep Learning

KSA 2027-Spring

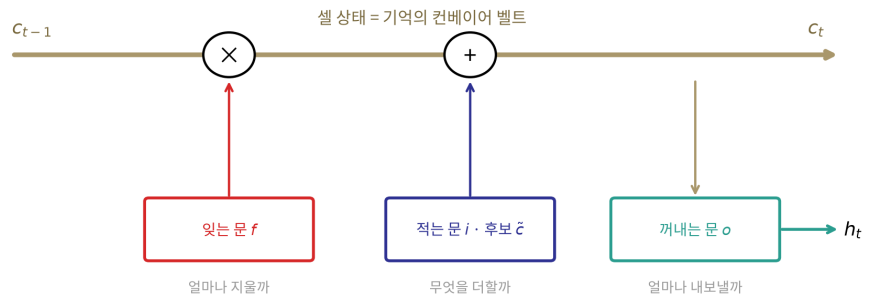
Sumin Han (suminhan@ksa.hs.kr)

오늘의 질문: 100걸음 뒤 기억은 얼마나 남을까?



기본 RNN 처럼 걸음마다 기억에 0.9 를 곱하면 $0.9^{100} \approx$ **0.003%** 사실상 0

LSTM: 기억을 실어 나르는 컨베이어 벨트



입력: 지금 단어 x_t 와 이전 출력 h_{t-1} → 세 문 모두 0~1 (시그모이드)

셀 상태 c_t 는 **더하기**로 이어진다 — Hochreiter & Schmidhuber (1997)

세 개의 **문 (gate)** 이 지우기 · 적기 · 꺼내기를 조절 — 잊는 문은 Gers, Schmidhuber, Cummins (2000) 가 추가

식으로

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t]) \qquad \mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t]) \qquad \tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

σ: $0 \sim 1$ 문은 “얼마나 열지”를 칸마다 정한다 (0 = 닫힘, 1 = 활짝)

$\odot$ 칸별 곱 편향은 생략해서 썼다

손으로 한 번 (기억 한 칸)

이전 기억 $c_{t-1} = 2$ 후보 $\tilde{c} = 0.8$

잊는 문 $f = 0.9$ 적는 문 $i = 0.5$ 꺼내는 문 $o = 0.7$

$$c_t = \underbrace{0.9 \times 2}_{\text{keep 90\%}} + \underbrace{0.5 \times 0.8}_{\text{write}} = 1.8 + 0.4 = \mathbf{2.2}$$

$$h_t = 0.7 \times \tanh(2.2) = 0.7 \times 0.976 = \mathbf{0.68}$$

기억의 90% 를 지키고, 새 내용을 절반만 더하고, 70% 만 꺼내 보였다

왜 오래 기억할 수 있나

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t \qquad \text{(기본 RNN: } \frac{\partial h_t}{\partial h_{t-1}} = W_h \cdot \tanh'(\cdot)\text{)}$$

셀 상태 경로에는 행렬도 tanh 도 없다 걸음마다 잊는 문 f **하나만** 곱해진다

기억해야 할 때 $f \approx 1$ 로 학습하면 $0.999^{100} \approx$ **90%** 기울기도 그대로 전달

논문: 1,000걸음 넘게 떨어진 정보도 배울 수 있음을 보였다 ResNet 의 지름길 (9주차) 과 같은 생각

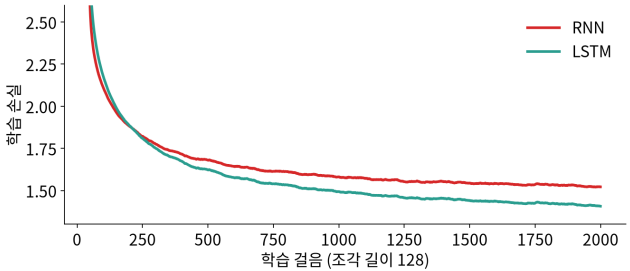
문이 하는 일: 예

"The **cat**, which ate all the fish in the kitchen, **was** full"

cat	적는 문 열림	기억 칸 하나에 "주어는 단수" 를 적는다
which ate $\cdots$ kitchen	잊는 문 ≈ 1	그 칸을 지키며 지나간다
was	꺼내는 문 열림	"단수" 를 꺼내 was 를 고르게 한다
(문장 끝)	잊는 문 ≈ 0	다음 문장을 위해 지운다

실제로 학습된 LSTM 에서 이렇게 해석되는 칸들이 발견된다 (예: 따옴표 안인지, 줄 길이) Karpathy 외 (2015)

실험: 셰익스피어 글자 모델, RNN vs LSTM



검증 손실 (뒤 10% 글)	조각 64	조각 128
RNN	1.74	1.71
LSTM	**1.64**	**1.61**

더 간단한 친척: GRU

GRU (Gated Recurrent Unit, Cho 외 2014): 문 **두 개**, 셀 상태와 출력을 하나로

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t$$

z: 갱신 문 (지키는 만큼 덜 적는다) 가중치가 LSTM 의 $\frac{3}{4}$

자세히는 다음 시간 PyTorch: `nn.LSTM`, `nn.GRU`

LSTM 이 활약한 곳

2014	Seq2Seq 번역 (Sutskever 외): LSTM 4층, 영어 $\rightarrow$ 프랑스어
2015 $\sim$ 2016	음성 인식 · 구글 번역 (GNMT) 의 주력
2017 $\sim$	트랜스포머로 넘어간다: 모든 단어를 **한꺼번에** 본다

그래도: 작은 시계열, 실시간 처리 (한 칸씩 들어오는 데이터) 에는 지금도 쓰인다

문제풀이

1. $c_{t-1} = 4$, $f = 0.5$, $i = 1$, $\tilde{c} = -1$ 이면 c_t 는? $o = 1$ 이면 h_t 는? ($\tanh 1 \approx 0.76$)
2. 잊는 문이 계속 0.99 면 100걸음 뒤 기억은 몇 % 남나? ($0.99^{100} \approx 0.366$)
3. 입력 100칸, 은닉 128칸인 LSTM 층의 가중치 수는? (문 · 후보 4묶음, 각각 $[\mathbf{h}, \mathbf{x}]$ 에 곱하고 편향)
4. 문장이 끝날 때 잊는 문이 0 에 가까워지는 것이 왜 쓸모 있을까?

실습

1. LSTM 한 칸 직접 구현 → 손 계산 2.2 · 0.68, `nn.LSTMCell` 과 비교
2. 셰익스피어 글자 모델: `nn.RNN` vs `nn.LSTM` vs `nn.GRU`
3. 가중치 수 비교, 학습된 LSTM 으로 글 써 보기

Tiny Shakespeare (Karpathy, char-rnn)

▶ Colab에서 실습 노트북 열기

정리

기본 RNN: 걸음마다 곱해져 기억 · 기울기가 사라진다 ($0.9^{100} \approx 0.003\%$)

LSTM: 셀 상태 $c_t = f \odot c_{t-1} + i \odot \tilde{c}$ **더하기**로 이어지는 기억

세 문: 잊기 f · 적기 i · 꺼내기 o ($0 \sim 1$) $f \approx 1$ 이면 오래 간다

셰익스피어 검증 손실 1.71 $\rightarrow$ 1.61 다음: 문 두 개인 GRU, 그리고 Seq2Seq